

MonteCristo

Manual del equipo

Cómo trabajar un ticket de principio a fin:
del repositorio a Arena.ai, y de Arena.ai al pull request

Lee esto antes de tocar nada

Este manual explica el único flujo de trabajo que vamos a usar todos. Si cada quien trabaja a su manera, se nos van a empalmar los cambios y vamos a perder tiempo arreglando conflictos. Son 10 pasos y la primera vez te toma unos 20 minutos; después son 5.

La regla más importante

Los pull requests SIEMPRE se abren desde GitHub, con tu propia cuenta. NUNCA le pidas a Arena.ai que haga commits ni que suba el pull request. El trabajo lo subimos nosotros: así el historial muestra quién hizo qué y el profesor ve el esfuerzo repartido entre los cuatro.

Contenido

Paso	Qué vas a hacer
Quién hace qué	El equipo, los roles y qué tickets le tocan a cada quien
Antes de empezar	Lo que necesitas instalado y las cuentas que debes tener
Paso 1	Aceptar la invitación al repositorio
Paso 2	Clonar el repositorio en tu computadora
Paso 3	Conectar Arena.ai con GitHub y abrir el repositorio
Paso 4	Elegir tu ticket en Jira y copiar su prompt
Paso 5	Pegar el prompt en Arena.ai y revisar lo que te dio
Paso 6	Descargar el .zip (solo el código del ticket)
Paso 7	Copiar los archivos a tu copia y probar que funcione
Paso 8	Crear tu rama y hacer commit
Paso 9	Subir la rama y abrir el pull request en GitHub
Paso 10	Que un compañero lo revise y se haga el merge
Ejemplo completo	Un ticket de principio a fin, con los comandos reales
Problemas comunes	Qué hacer cuando algo sale mal
Reglas del equipo	Lista para revisar antes de pedir el merge
Anexos	Guías de lo que se hace fuera de Arena: diagramas, Neon, Render y pruebas

Quién hace qué

Los 40 tickets ya están repartidos según el rol de cada quien. En Jira vienen asignados y en el archivo **PROMPTS-POR-TICKET.txt** cada ticket dice arriba quién es el responsable.

Integrante	Rol	De qué se encarga	Tickets
Luis Enrique Rodríguez González	Product Owner y líder	Visión del producto, backlog, sprints y enlace con el profesor	1, 2, 5, 16, 40
Castell Guzmán Elian Avishayr	Backend · motor de juegos	Reglas de los juegos, azar del servidor, cobro y pago de apuestas	19 a 28
Alan Emmanuel Oseguera Michel	Backend · API y datos	Servidor, API, base de datos en Neon, saldos y despliegue	6 a 14, 39
Héctor Jaime Navarro Guillén	Frontend	Maquetas, estilos, pantallas, animaciones y chat	29 a 37
Josué Ángel Carrillo Cruz	Documentación y QA	Diagramas, módulo de salas, plan de pruebas y entregables	3, 4, 15, 17, 18, 38

En qué orden conviene trabajar

No todos pueden empezar al mismo tiempo: hay tickets que dependen de otros. Esta es la secuencia sana:

- **Semana 1.** Luis saca los requerimientos y las reglas (1, 2); Josué, los diagramas (3, 4). Alan arranca con la base en Neon y el servidor (6 a 12). Héctor va haciendo las maquetas y la hoja de estilos (29).
- **Semana 2.** Alan termina API y saldos (13, 14); Josué hace el módulo de salas (15, 17, 18); Luis el código de sala (16); Héctor, inicio y formulario (30, 31).
- **Semana 3.** Elian arranca el motor: ciclo de ronda, azar, apuestas y turnos (19 a 22). Héctor hace lobby y mesa (32, 33).
- **Semana 4.** Elian saca los juegos (23 a 28). Héctor, los botones de apuesta, los mensajes y las animaciones (34, 35, 36).
- **Semana 5.** Héctor cierra con el chat (37). Josué corre el plan de pruebas con todo el equipo (38). Alan publica en Render (39).
- **Semana 6.** Luis arma la presentación (40) y se ensaya entre todos.

Si te quedas esperando a alguien

Avísale en el grupo y toma otro de tus tickets que no dependa de eso, o échale la mano al que va retrasado. Lo que no se vale es quedarse parado tres días sin decir nada: el profesor va a ver las fechas en Jira y en los commits.

Antes de empezar

Necesitas tener esto listo. Si te falta algo, instálalo ahora: después, a media tarea, estorba más.

Qué	Para qué	Dónde
Cuenta de GitHub	Para acceder al repositorio y abrir los pull requests	github.com
Cuenta de Arena.ai	Para generar el código de tu ticket	arena.ai
Git instalado	Para bajar y subir cambios desde tu computadora	git-scm.com
Node.js 18 o más	Para correr el proyecto y probarlo	nodejs.org
Un editor de código	Para revisar y ajustar lo que te dieron	VS Code, por ejemplo
Acceso a Jira	Para ver qué ticket te tocó	El enlace que pasó el equipo

Comprueba que Git y Node estén bien instalados abriendo una terminal (CMD o PowerShell en Windows, Terminal en Mac) y escribiendo:

```
git --version
node --version
npm --version
```

Si los tres responden con un número de versión, estás listo. Si alguno dice que el comando no existe, falta instalarlo.

Cómo bajar este manual y los prompts

Todos los archivos del kit viven en el repositorio, así que siempre los puedes bajar desde GitHub aunque no tengas ningún enlace a la mano.

Bajar un archivo suelto (el PDF o el TXT)

- 1 Entra al repositorio en GitHub y abre la carpeta donde están los archivos del kit.
- 2 Da clic en el archivo que quieres (por ejemplo **PROMPTS-POR-TICKET.txt**).
- 3 Arriba a la derecha del archivo hay un botón de descarga (**Download raw file**, el icono de la flecha hacia abajo). Dale clic y se guarda en tu computadora.
- 4 Si es un PDF, GitHub lo muestra en pantalla: usa el mismo botón de descarga.

Bajar todo el proyecto de una vez

- 1 En la página principal del repositorio, botón verde **Code** → **Download ZIP**.
- 2 O, si ya lo clonaste, los archivos ya están en tu carpeta: solo haz `git pull`.

Si te pasan un enlace de vista previa y no te deja descargar

Las vistas previas se abren dentro de un marco que a veces bloquea las descargas. Busca el botón de abrir en una pestaña nueva (el icono de la flecha diagonal) y descarga desde ahí. Si aun así no baja, usa GitHub: nunca falla.

Paso 1. Aceptar la invitación al repositorio

- 1 Quien creó el repositorio te invita desde **Settings** → **Collaborators** → **Add people**, escribiendo tu usuario de GitHub.
- 2 Te llega un correo con el asunto “... *invited you to collaborate*”. También aparece en github.com/notifications.
- 3 Abre el correo y da clic en **Accept invitation**.
- 4 Entra al repositorio: si ya puedes ver el botón verde **Code** y la pestaña **Pull requests**, quedó listo.

Si no te llegó el correo

Pide que te manden el enlace directo del repositorio y ábrelo estando con tu sesión de GitHub iniciada. La invitación aparece hasta arriba de la página.

Paso 2. Clonar el repositorio en tu computadora

Clonar es bajar una copia del proyecto a tu máquina. Se hace una sola vez.

- 1 En el repositorio, da clic en el botón verde **Code** y copia la dirección HTTPS.
- 2 Abre una terminal en la carpeta donde guardas tus proyectos (por ejemplo Documentos).
- 3 Escribe los comandos de abajo, cambiando la dirección por la de nuestro repositorio.

```
cd Documentos
git clone https://github.com/USUARIO/montecristo.git
cd montecristo
npm install
```

A partir de aquí, **siempre** trabajas dentro de esa carpeta. Cada vez que te sientes a trabajar, lo primero es actualizar tu copia:

```
git checkout main
git pull
```

No te saltes el git pull

Si trabajas sobre una copia vieja, el código que te dé Arena va a chocar con lo que tus compañeros ya subieron, y vas a pelearte con conflictos que no tenías por qué tener.

Paso 3. Conectar Arena.ai con GitHub y abrir el repositorio

- 1 Entra a **arena.ai** e inicia sesión.
- 2 Busca la opción de conectar GitHub (normalmente en la configuración de la cuenta o cuando abres un proyecto nuevo, con el botón **Connect GitHub**).
- 3 GitHub te va a pedir autorización: acepta y, si te deja elegir, selecciona el repositorio de MonteCristo. Con permiso solo a ese repositorio es suficiente.
- 4 Ya conectado, abre el repositorio en Arena. Debe cargarte los archivos del proyecto: así el código que te genere va a encajar con el que ya existe.
- 5 Confirma que esté en la rama **main** y actualizada antes de empezar.

¿Por qué conectarlo si al final bajo un zip?

Porque con el repositorio abierto, Arena ve cómo están escritos los demás archivos y respeta los nombres, las carpetas y el estilo del proyecto. Sin eso, te inventa una estructura distinta y luego no embona.

Lo que NO vas a usar de Arena

Aunque Arena pueda crear ramas, hacer commits y abrir pull requests, nosotros no vamos a usar nada de eso. Solo lo usamos para generar el código y descargar el zip. Todo lo demás lo haces tú desde tu computadora y desde GitHub.

Paso 4. Elegir tu ticket y copiar su prompt

- 1 Entra a Jira y busca el ticket que te tocó. Anota su número (por ejemplo, el 24).
- 2 Revisa que los tickets de los que depende ya estén terminados y fusionados en main. Si el tuyo es el de la ruleta y todavía no existe el ciclo de la ronda, espérate o avisa al equipo.
- 3 Abre el archivo **PROMPTS-POR-TICKET.txt** que viene en el kit.
- 4 Busca tu número de ticket y copia **todo** lo que está entre **[COPIA DESDE AQUI]** y **[HASTA AQUI]**, sin quitarle ni agregarle nada.

Ejemplo

A Luis le tocó el ticket 24 (Juego de dados). Abre el .txt, busca "TICKET 24 - Juego de dados", selecciona desde **[COPIA DESDE AQUI]** hasta **[HASTA AQUI]** y lo copia con Ctrl+C.

No improvises el prompt

El prompt ya trae el contexto del proyecto, las tecnologías, los pagos de cada juego y las reglas de entrega. Si escribes el tuyo "con tus palabras", te va a dar algo distinto a lo de tus compañeros y el proyecto va a quedar hecho de parches.

Paso 5. Pegar el prompt y revisar lo que te dio

- 1 Con el repositorio ya abierto en Arena, pega el prompt en el chat y envíalo.
- 2 Espera a que termine y **lee** lo que hizo. No lo bajes a ciegas.
- 3 Revisa tres cosas: que sean los archivos de tu ticket y nada más, que las rutas de las carpetas coincidan con las del proyecto, y que no haya tocado archivos de otros compañeros.
- 4 Si algo no cuadra, corrígelo en el mismo chat (abajo hay frases listas para eso).
- 5 Si no entiendes una parte del código, pídele que te la explique línea por línea. Lo vas a necesitar en la exposición.

Frases útiles para corregir, tal cual se pueden pegar:

Si pasa esto...	Respóndele esto
Te generó documentación o README de más	"Quítame la documentación, solo quiero los archivos de código de este ticket en el zip."
Te adelantó trabajo de otros tickets	"Eso es de otro ticket, déjame únicamente lo que pedí."
Hizo commit, rama o pull request	"No hagas commits ni pull requests, yo los subo. Solo dame el zip con los archivos."
Cambió archivos que no eran	"No toques los demás archivos del repositorio, solo los de este ticket."
No te dio el zip	"Déjame los archivos en un .zip para descargar, respetando las rutas de carpetas."
No le entiendes al código	"Explícame este archivo línea por línea como si fuera principiante."

Paso 6. Descargar el .zip

- 1 Pídelo o búscalo en la respuesta: debe ser un archivo **.zip** para descargar.
- 2 Ábrelo antes de copiar nada y revisa que adentro vengan **solo** los archivos de tu ticket.
- 3 Si trae documentación, diagramas o archivos que no pediste, **no los copies**: usa solo lo que corresponde a tu ticket.

Ejemplo de un zip correcto (ticket 24, dados)

src/juegos/dados.js · public/juegos/dados.js · public/mesa.html (modificado)

Si además viniera docs/dados.md o un README.md, eso sobra: no lo copies.

Por qué solo el código

La documentación del proyecto ya está repartida en sus propios tickets. Si cada quien sube su versión de la documentación, terminamos con cinco archivos que dicen cosas distintas.

Paso 7. Copiar los archivos y probar que funcione

- 1 Descomprime el zip en una carpeta aparte (en Descargas, por ejemplo).
- 2 Copia los archivos a tu copia del repositorio **respetando las rutas**: si el archivo venía en src/juegos/, va en src/juegos/ del proyecto.
- 3 Corre el proyecto y Pruébalo de verdad, no solo que “arranque”.

```
npm install      # solo si tu ticket agregó alguna librería nueva
npm start
# abre http://localhost:3000 en el navegador
```

Revisa que lo de tu ticket funcione y que **lo que ya existía siga funcionando**. Si rompiste algo de otro compañero, arréglalo antes de subir.

```
git status      # te muestra qué archivos cambiaste
git diff        # te muestra exactamente qué cambió en cada uno
```

Dale una pasada tú mismo

Cambia algún nombre de variable que no te guste, acomoda un comentario, borra lo que sobre. Además de que queda mejor, te obliga a leer el código que vas a tener que defender frente al profesor.

Paso 8. Crear tu rama y hacer commit

Una rama por ticket. El nombre lleva la clave del ticket de Jira para que quede ligado solo.

```
git checkout main
git pull
git checkout -b MC-24-juego-de-dados

git add .
git commit -m "MC-24 Agregar la logica del juego de dados"
git commit -m "MC-24 Agregar los botones de apuesta de dados"
```

Haz **varios commits chicos** mientras avanzas, no uno gigante al final. Escribe los mensajes tú, en español y explicando qué hiciste.

Mensaje de commit	¿Sirve?
MC-24 Agregar la logica del juego de dados	Sí: dice qué se hizo y en qué ticket
MC-24 Arreglar el pago del numero exacto	Sí: concreto y entendible
cambios	No: no dice nada
asdasd / update / fix	No: se nota el relleno
Agregar todo el proyecto	No: es un commit gigante de último día

Paso 9. Subir tu rama y abrir el pull request en GitHub

Esto se hace en GitHub, no en Arena.ai

Aquí es donde queda registrado que el trabajo es tuyo. Si el pull request lo abre una herramienta en lugar de ti, el historial del repositorio deja de reflejar quién hizo qué, y es justo lo que no queremos.

- 1 Sube tu rama con el comando de abajo.
- 2 Entra al repositorio en GitHub. Va a aparecer un aviso amarillo con el botón **Compare & pull request**: dale clic. (Si no sale, ve a la pestaña **Pull requests** → **New pull request** y elige tu rama.)
- 3 Revisa que la base sea **main** y que la rama que comparas sea la tuya.
- 4 Escribe el título con la clave del ticket y un resumen corto; en la descripción, qué hiciste, qué archivos tocaste y cómo probarlo.
- 5 Dale **Create pull request** y avísale al equipo para que alguien lo revise.

```
git push -u origin MC-24-juego-de-dados
```

Ejemplo de pull request bien escrito

Título: MC-24 Juego de dados

Descripción:

Agregué la lógica del juego de dados en src/juegos/dados.js y los botones de apuesta en la mesa. Se puede apostar a bajo, alto o a un número exacto; los pagos son x2 y x6 como dice el documento de reglas.

Para probarlo: npm start, crear una sala de dados, apostar 10 fichas a “bajo” y lanzar la ronda. El resultado lo decide el servidor.

Cierra el ticket MC-24.

Paso 10. Revisión y merge

- 1 Un compañero **distinto** al autor abre el pull request, lee los cambios en la pestaña **Files changed** y prueba que funcione en su computadora.
- 2 Deja al menos un comentario real: una duda, una mejora o un “ya lo probé y funciona”.
- 3 Si hay que corregir algo, el autor hace más commits en la misma rama y se actualiza solo.
- 4 Cuando esté bien, se da **Merge pull request** y luego **Delete branch**.
- 5 El autor actualiza su copia local y pasa al siguiente ticket.

```
git checkout main
git pull
git branch -d MC-24-juego-de-dados
```

Repártanse las revisiones

Que no sea siempre el mismo quien revisa. Es la forma más fácil de que los cuatro conozcan todo el proyecto y no solo su pedazo, que es exactamente lo que el profesor va a preguntar en la exposición.

Ejemplo completo, de principio a fin

Luis tiene el ticket **MC-24, Juego de dados**. Esto es todo lo que hace:

#	Qué hace Luis
1	Abre Jira y confirma que el ticket 19 (ciclo de la ronda) ya está fusionado en main.
2	Abre su terminal: <code>cd montecristo, git checkout main, git pull</code> .
3	Entra a arena.ai con el repositorio de MonteCristo conectado y abierto.
4	Abre PROMPTS-POR-TICKET.txt, busca el TICKET 24 y copia el bloque completo.
5	Lo pega en el chat de Arena y espera.
6	Revisa la respuesta: son dos archivos de dados y un cambio en la mesa. Correcto.
7	Descarga el zip y lo descomprime en Descargas.
8	Copia <code>src/juegos/dados.js</code> y <code>public/juegos/dados.js</code> a su copia del repositorio.
9	Corre <code>npm start</code> , crea una sala de dados y juega una ronda. Funciona.
10	<code>git checkout -b MC-24-juego-de-dados</code>
11	<code>git add .</code> y dos commits con mensajes claros.
12	<code>git push -u origin MC-24-juego-de-dados</code>
13	En GitHub da Compare & pull request , escribe el título y la descripción, y lo crea.
14	Avisa en el grupo. Ana lo revisa, prueba y comenta "funciona, solo acomoda el margen".
15	Luis hace un commit más con el ajuste. Ana da merge y borra la rama.
16	Luis mueve el ticket a Terminado en Jira y toma el siguiente.

Problemas comunes

Problema	Qué hacer
"Permission denied" o no te deja hacer push	Todavía no aceptas la invitación al repositorio, o estás con otra cuenta de GitHub. Revisa el Paso 1.
GitHub te pide usuario y contraseña y no la acepta	GitHub ya no acepta contraseñas: usa un token personal (Settings → Developer settings → Personal access tokens) o instala GitHub Desktop.
"Your branch is behind main"	Alguien subió cambios antes que tú. Haz <code>git checkout main</code> , <code>git pull</code> , vuelve a tu rama y <code>git merge main</code> .
El pull request dice que hay conflictos	Dos personas tocaron el mismo archivo. Avísale al otro, abran el archivo juntos y dejen la versión correcta. No borres el trabajo del otro sin avisar.
El código del zip no embona con el proyecto	Casi siempre es porque trabajaste sobre una copia vieja. Haz <code>git pull</code> , vuelve a pegar el prompt y agrégale: "Ten en cuenta el código que ya existe en el repositorio".
Arena te dio documentación que no pediste	Ignórala: copia solo los archivos de código. O pídele que te regenere el zip sin ella.
Te equivocaste de rama y commiteaste en main	No hagas push. Avisa al equipo y pide ayuda: se arregla fácil, pero mejor entre dos.
npm start truena después de copiar el zip	Lee el error completo: casi siempre falta un <code>npm install</code> o un archivo que no copiaste. Pégale el error a Arena y pídele que te explique la causa.

Reglas del equipo

Antes de pedir que te hagan merge, revisa que cumplas todo esto:

- Hice `git pull` de main antes de empezar.
- Trabajé en mi propia rama, con la clave del ticket en el nombre.
- El zip que usé traía solo el código de mi ticket, sin documentación de más.
- Probé que mi parte funciona y que no rompí lo que ya existía.
- Mis commits son varios, chicos y con mensajes escritos por mí.
- El pull request lo abrí yo desde GitHub, no desde Arena.ai.
- Escribí en la descripción qué hice y cómo probarlo.
- Puedo explicar línea por línea lo que estoy subiendo.

Las dos cosas que no se negocian

1. Nunca hagas commits ni pull requests desde Arena.ai: los subimos nosotros desde nuestras cuentas.
2. Nunca subas código que no puedas explicar. Si no le entiendes, pide que te lo expliquen antes de subirlo; en la exposición no va a haber a quién preguntarle.

Una nota sobre la IA y el profesor

Usar IA para planear y resolver dudas está bien y hoy es lo normal; lo que no se vale es entregar algo que nadie del equipo entiende. Si el profesor permite el uso de IA declarándolo, lo más sano es anotarlo en el README: cuesta mucho menos que te lo descubran. Lo que de verdad defiende el proyecto es que ustedes puedan explicar cada decisión y que el historial muestre el trabajo repartido entre los cuatro.

Anexos: lo que se hace fuera de Arena

Hay tickets que no se terminan en el chat: hay que entrar a otra página o abrir otro programa. Aquí están esos pasos explicados. En el archivo **PROMPTS-POR-TICKET.txt** cada uno de esos tickets trae el aviso “OJO: ESTE TICKET NO SE TERMINA DENTRO DE ARENA” con los mismos pasos.

Anexo	Para qué tickets	Dónde hay que entrar
A. Exportar un diagrama a imagen	3 y 4	mermaid.live
B. Crear la base de datos	6	neon.tech
C. Correr los scripts SQL	7, 8 y 9	SQL Editor de Neon
D. Probar entre varias computadoras	38	Tu red local o la página publicada
E. Publicar la página	39	render.com
F. Armar las diapositivas	40	Canva, PowerPoint o Google Slides

Anexo A. Exportar un diagrama a imagen (tickets 3 y 4)

En estos tickets el prompt ya le pide a Arena que genere la imagen en PNG. Si te la dio, solo cópiala a la carpeta docs/ y ya está. Estos pasos son por si la quieres rehacer tú o cambiarle algo.

- 1 Entra a **<https://mermaid.live>** (no hay que registrarse).
- 2 Borra el ejemplo del panel izquierdo con Ctrl+A y Suprimir.
- 3 Abre el archivo .md que te dio Arena y copia **solo** lo que está dentro del bloque de código (empieza con `erDiagram` o `flowchart`), sin las comillas invertidas.
- 4 Pégalo en el panel izquierdo: el dibujo aparece a la derecha.
- 5 Clic en **Actions** (abajo del panel derecho) y luego en **PNG**.
- 6 Renombra la imagen y cópiala a la carpeta docs/ del repositorio.

Anexo B. Crear la base de datos en Neon (ticket 6)

Nuestra base de datos es **PostgreSQL en Neon**, que vive en la nube: no hay que instalar nada en la computadora y el plan gratuito no pide tarjeta.

- 1 Entra a **<https://neon.tech>** y da clic en **Sign up**. Puedes entrar con tu cuenta de GitHub.
- 2 Crea un proyecto: nómbralo **montecristo**, deja la versión de PostgreSQL por defecto y elige la región más cercana (por ejemplo US East). Clic en **Create project**.
- 3 Al terminar aparece el recuadro **Connection string**, con algo como `postgresql://usuario:clave@ep-algo.neon.tech/neondb?sslmode=require`. Cópiala completa.
- 4 Si cerraste el recuadro, la encuentras en el **Dashboard** del proyecto, en **Connection Details**.
- 5 En la carpeta del proyecto crea el archivo .env y pega adentro: `DATABASE_URL="la cadena que copiaste"`

Una sola base para todo el equipo

Solo una persona crea el proyecto en Neon y les pasa la cadena a los demás **por privado**. Así los cuatro trabajan sobre la misma base y ven los mismos datos. Y ojo: en el plan gratis la base se duerme si nadie la usa, así que la primera consulta después de un rato tarda unos segundos. No está descompuesta.

El archivo .env no se sube nunca

La cadena de conexión trae usuario y contraseña. En el repositorio solo va `.env.example` con la variable vacía, y `.env` debe estar listado en el `.gitignore`. Si subes la cadena real, cualquiera que vea el repo puede entrar a la base.

Anexo C. Correr los scripts SQL (tickets 7, 8 y 9)

- 1 Entra a <https://console.neon.tech> y abre el proyecto montecristo.
- 2 En el menú de la izquierda, clic en **SQL Editor**.
- 3 Abre el archivo .sql que te dio Arena, copia todo y pégalo en el editor.
- 4 Clic en **Run** (o Ctrl+Enter). El resultado sale abajo.
- 5 Para comprobar, entra a **Tables** en el menú de la izquierda: ahí debe aparecer la tabla con sus campos.

El orden importa

Primero jugadores (ticket 7), luego salas (8) y al final partidas (9). Las tablas de salas y partidas apuntan a jugadores con llaves foráneas: si las corres antes, marca error de *foreign key*. Y si el script trae AUTO_INCREMENT, es sintaxis de MySQL: en PostgreSQL va SERIAL. Pídele a Arena que lo corrija.

Anexo D. Probar entre varias computadoras (ticket 38)

- 1 Conéctense todos a la misma red (el wifi de la escuela o un celular compartiendo datos).
- 2 Quien tenga el proyecto corriendo busca su IP local: en Windows, terminal y ipconfig; es la que dice *Dirección IPv4* (algo como 192.168.1.75).
- 3 Los demás entran desde su navegador a `http://ESA-IP:3000`
- 4 Si no carga, es el firewall de Windows: cuando salga el aviso, den **Permitir acceso** para redes privadas.
- 5 Opción más cómoda: hagan las pruebas directo sobre la página ya publicada (ticket 39).

Anexo E. Publicar la página (ticket 39)

La base de datos ya está en la nube desde el ticket 6, así que solo falta el servidor.

Antes de empezar

- 1 Entra a **<https://console.neon.tech>** y copia otra vez la cadena de conexión del **Dashboard**: la vas a necesitar en Render.
- 2 En el **SQL Editor**, revisa que ya existan las tres tablas (jugadores, salas y partidas). Si no, corre los scripts de los tickets 7, 8 y 9.

Servidor en Render

- 1 Entra a **<https://render.com>** e inicia sesión con GitHub.
- 2 **New +** → **Web Service** → autoriza y elige el repositorio de MonteCristo.
- 3 Llena: **Name** montecristo · **Runtime** Node · **Build Command** `npm install` · **Start Command** `npm start` · **Instance Type** Free.
- 4 En **Environment Variables** agrega una sola variable: **DATABASE_URL**, con la cadena de conexión de Neon (la que termina en `?sslmode=require`).
- 5 **Create Web Service** y espera a que el log diga **Live**.
- 6 Arriba aparece la dirección pública (algo como `https://montecristo.onrender.com`). Anótenla en el README.

Para el día de la exposición

El plan gratis duerme la página tras 15 minutos sin visitas, y la primera carga después tarda cerca de un minuto. Abren la página 5 minutos antes de exponer para que ya esté despierta.

Anexo F. Armar las diapositivas (ticket 40)

- 1 Entra a **<https://www.canva.com>** (o usa PowerPoint o Google Slides).
- 2 Busca **Presentación** y elige una plantilla oscura y sencilla.
- 3 Pasa el guion que te dio Arena diapositiva por diapositiva: título arriba y máximo 4 puntos cortos. Nada de párrafos.
- 4 Mete los diagramas de los tickets 3 y 4 y capturas de la página funcionando (Win+Shift+S).
- 5 Descárgala en PDF y súbela a docs/, por si falla el internet.
- 6 Ensayen una vez con reloj: 10 minutos entre cuatro se van rapidísimo.